Building a Production AI Agent from Scratch

Local File Reader & Summarizer MVP

Software Engineering Team

May 2026

Contents

1 Introduction & Fundamentals **2**
1.1 LLM Fundamentals 2
1.2 The ReAct Loop 2
1.3 Function Calling 2

2 Project Architecture **3**
2.1 Clean Architecture Layers 3
2.2 Dependency Management 3

3 Core Implementation **4**
3.1 The Domain Model 4
3.2 The Agentic Loop 4

4 Security & Security Patterns **6**
4.1 Path Traversal Prevention 6
4.2 Secret Management 6

5 Conclusion **7**

Chapter 1

Introduction & Fundamentals

1.1 LLM Fundamentals

Large Language Models (LLMs) like Claude are fundamentally **stateless**. They do not retain memory between independent API calls. To maintain a conversation, the entire history must be sent back to the model with every new request.

In the Anthropic SDK, this is managed via the `messages` list, which alternates between roles:

- `user`: The human participant.
- `assistant`: The Claude model.

1.2 The ReAct Loop

ReAct (Reasoning and Acting) is a paradigm that allows LLMs to interact with external tools. The process follows a loop:

1. **Reasoning**: The model analyzes the user request and decides whether it needs external information.
2. **Acting**: If the model decides to use a tool, it returns a `stop_reason == "tool_use"`.
3. **Execution**: The application executes the requested Python function.
4. **Observation**: The application feeds the result back to the model as a `tool_result` within a `user` turn.
5. **Iteration**: The loop repeats until the model provides a final answer (`stop_reason == "end_turn"`).

1.3 Function Calling

Function calling (or Tool Use) involves providing Claude with a JSON Schema that describes a Python function's name, purpose, and parameters. Claude uses this schema to format a request for your code to execute.

Chapter 2

Project Architecture

2.1 Clean Architecture Layers

The project follows a 4-layer Clean Architecture to ensure separation of concerns:

1. **Domain Layer**: Core business entities (dataclasses and exceptions) with zero external dependencies.
2. **Infrastructure Layer**: Real-world interactions (file system access, session stores).
3. **Application Layer**: Orchestrates the business logic and the agentic loop.
4. **Presentation Layer**: FastAPI routers and Pydantic schemas for HTTP communication.

2.2 Dependency Management

The project uses `uv`, a fast Python package manager. The `pyproject.toml` defines dependencies including `fastapi`, `anthropic`, and `pydantic-settings`.

Chapter 3

Core Implementation

3.1 The Domain Model

The `AgentSession` dataclass represents a conversation thread.

```python
@dataclass
class AgentSession:
    session_id: str
    messages: list[dict[str, Any]] = field(default_factory=list)

    def add_user_message(self, content: str) -> None:
        self.messages.append({"role": "user", "content": content})

    def add_assistant_message(self, content: Any) -> None:
        self.messages.append({"role": "assistant", "content": content})

    def add_tool_results(self, tool_results: list[dict[str, Any]]) -> None:
        self.messages.append({"role": "user", "content": tool_results})
```

Listing 3.1: app/domain/models.py

3.2 The Agentic Loop

The `AgentService` manages the iterative tool-use cycle.

```python
async def run(self, session: AgentSession) -> str:
    for iteration in range(MAX_LOOP_ITERATIONS):
        response = await client.messages.create(
            model=settings.claude_model,
            tools=TOOL_DEFINITIONS,
            messages=session.messages,
        )

        if response.stop_reason == "end_turn":
            session.add_assistant_message(serialize(response.content))
            return extract_text(response)

        if response.stop_reason == "tool_use":
            session.add_assistant_message(serialize(response.content))
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    result = execute_tool(block.name, block.input)
                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
```

```
                        "content": result,
                    })
            session.add_tool_results(tool_results)
```

Listing 3.2: app/services/agent.py (Simplified)

Chapter 4

Security & Security Patterns

4.1 Path Traversal Prevention

To prevent unauthorized file access, the tool layer uses `Path.resolve()` followed by a parent check.

```
requested = (DOCS_DIR / file_name).resolve()
if DOCS_DIR not in requested.parents and requested != DOCS_DIR:
    return "Error: Access denied"
```

4.2 Secret Management

API keys are protected using Pydantic's `SecretStr`, ensuring they are never logged or exposed in JSON outputs.

Chapter 5

Conclusion

This architecture provides a robust, production-ready foundation for AI agents. By decoupling the LLM loop from the transport layer and enforcing strict security patterns, we build agents that are both capable and safe.